I2C Controller and Endpoint Device Model

Design Document - C Implementation

This document explains the simplified transaction-level I2C subsystem model. This describes the design, build the project, run the tests, and discuss future extensions.

1. Objective

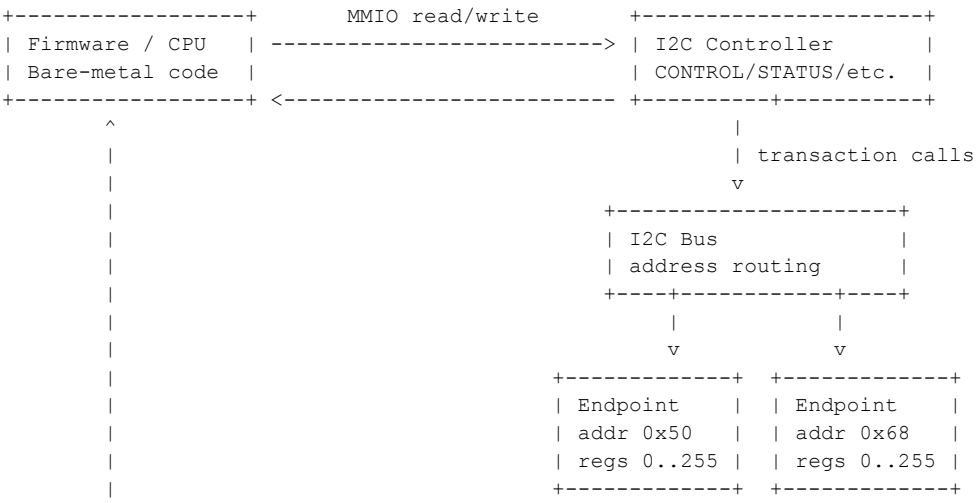
This Implementation is a simplified I2C subsystem model with an MMIO-exposed I2C controller/master, one or more endpoint/slave devices, and a bus abstraction that routes transactions to the matching slave address.

The model is functional and transaction-level. It intentionally does not model SDA/SCL electrical waveforms, clock stretching, arbitration, or cycle-accurate timing.

2. Requirements Mapping

Requirement	Implementation
MMIO controller registers	CONTROL, STATUS, TXDATA, RXDATA, and CMD are exposed as 32-bit MMIO words.
Commands	START, STOP, WRITE, and READ are supported through the CMD register.
Addressing	START consumes TXDATA as address byte: TXDATA[7:1] is 7-bit address; TXDATA[0] is R/W direction.
Read/write transactions	WRITE sends TXDATA to the endpoint. READ fetches one byte from endpoint into RXDATA.
Status reporting	BUSY, DONE, ERROR, and RX_VALID bits are implemented.
State machine	Controller maintains IDLE, START, SEND_ADDR, TRANSFER, STOP, and ERROR states.
Endpoint device address	Each endpoint has a fixed 7-bit address and responds only to that address.
Endpoint register map	Endpoint implements a 256-byte register map with Device ID, Status, and Data registers.
Bus abstraction	I2CBus stores endpoint pointers and routes lookups by 7-bit address.

3. High-Level Architecture



4. Source Layout

File	Purpose
include/i2c_model.h	Public types, register offsets, bit definitions, return codes, and APIs.
src/i2c_model.c	Controller, bus, endpoint, command handling, state updates, and MMIO read/write implementation.
src/demo.c	Simple example program that creates endpoints, reads/writes registers, and shows error behavior.
tests/test_i2c_model.c	Unit-style tests for normal transactions and error paths.
Makefile	Build targets for test, demo, and clean.
README.md	Quick build/run guide and concise design overview.

5. Controller Register Interface

Offset	Register	Access	Description
0x00	CONTROL	R/W	Enable/configuration register.
0x04	STATUS	R/W1C	Busy, done, error, and RX valid status.
0x08	TXDATA	R/W	Address byte or transmit data byte. Only low 8 bits are used.
0x0C	RXDATA	R	Received data byte. Only low 8 bits are used.
0x10	CMD	W	Command register. Read returns zero in this model.

5.1 CONTROL Bits

Bit	Name	Meaning
0	ENABLE	Enables the I2C controller.

5.2 STATUS Bits

Bit	Name	Meaning
0	BUSY	A transaction is active between START and STOP.
1	DONE	Last command completed.
2	ERROR	An error occurred.
3	RX_VALID	RXDATA contains valid read data.

DONE, ERROR, and RX_VALID are sticky software-visible bits. Software can clear them by writing 1 to the corresponding STATUS bit. BUSY is live and cannot be cleared directly by software.

5.3 CMD Values

Value	Command	Description
1	START	Begin transaction or repeated START.
2	STOP	End active transaction.
3	WRITE	Send one byte from TXDATA to the selected endpoint.
4	READ	Read one byte from selected endpoint into RXDATA.

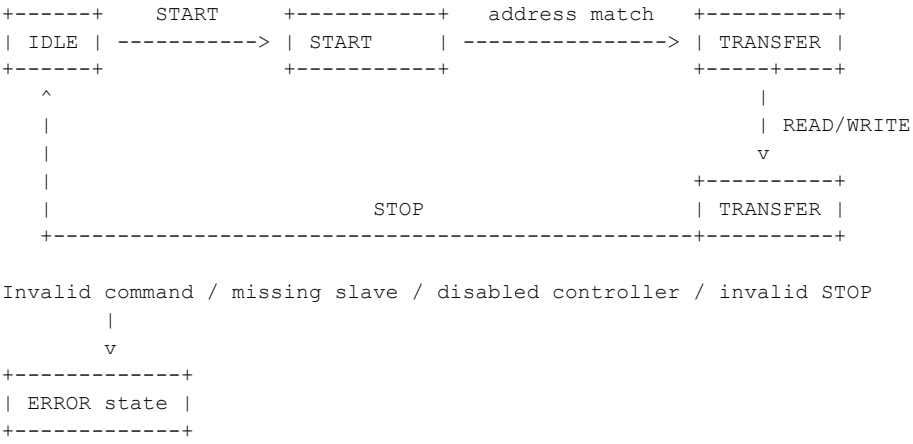
6. Addressing Convention

During START, TXDATA is interpreted as an I2C address byte:

```
TXDATA[7:1] = 7-bit slave address
TXDATA[0]   = direction bit: 0 = write, 1 = read

Example:
Slave address 0x50, write phase: TXDATA = 0xA0
Slave address 0x50, read phase:  TXDATA = 0xA1
```

7. Controller State Machine



State	Description
IDLE	No active transaction.
START	START command has been accepted.
SEND_ADDR	Transient address-resolution step inside START. Not normally visible externally.
TRANSFER	Transaction is active and READ/WRITE commands are valid depending on direction.
STOP	Transient stop-processing step.
ERROR	An invalid sequence, disabled access, missing slave, or invalid command occurred.

STOP is valid only when a transaction is active. STOP from IDLE or ERROR returns I2C_ERR_INVALID_SEQUENCE and sets the ERROR bit. This avoids masking firmware sequencing bugs.

8. Endpoint Device Model

Each endpoint has a fixed 7-bit address, a name, a device ID, a 256-byte register map, an internal register pointer, and small transaction state flags.

Offset	Register	Description
0x00	Device ID	Default initialized to the endpoint device_id.
0x01	Status	Generic endpoint status byte.
0x02	Data	Example data register used by tests.

The endpoint follows a common register-style I2C convention: the first WRITE byte after address phase sets the internal register pointer. Subsequent WRITE bytes update registers starting at that pointer. READ returns the byte at the current pointer and auto-increments the pointer.

Simplification: register 0x00 / Device ID is writable in this model. This keeps the endpoint generic. A production model could add per-register access permissions to make it read-only.

9. Bus Abstraction

The I2CBus object stores up to I2C_MAX_ENDPOINTS endpoint pointers. It rejects duplicate slave addresses and routes START address resolution to the matching endpoint.

```
START command:
1. Extract 7-bit address from TXDATA[7:1]
2. Extract direction from TXDATA[0]
3. Call i2c_bus_find(bus, address)
4. If endpoint exists: begin transaction and enter TRANSFER
5. If endpoint does not exist: set ERROR and return I2C_ERR_NO_SLAVE
```

10. Example: Read Device ID from Slave 0x50

```
1. TXDATA = (0x50 << 1) | 0   // address + write
2. CMD    = START
3. TXDATA = 0x00                // internal register pointer = Device ID register
4. CMD    = WRITE
5. TXDATA = (0x50 << 1) | 1     // repeated START + read
6. CMD    = START
7. CMD    = READ
8. RXDATA now contains Device ID
9. CMD    = STOP
```

11. Error Handling

Condition	Return Code	Visible STATUS
Command issued while controller disabled	I2C_ERR_DISABLED	ERROR + DONE
START to missing slave address	I2C_ERR_NO_SLAVE	ERROR + DONE
READ without active read transaction	I2C_ERR_INVALID_SEQUENCE	ERROR + DONE
WRITE without active write transaction	I2C_ERR_INVALID_SEQUENCE	ERROR + DONE
STOP from IDLE or ERROR	I2C_ERR_INVALID_SEQUENCE	ERROR + DONE
Invalid CMD value	I2C_ERR_INVALID_COMMAND	ERROR + DONE
Duplicate endpoint address attach	I2C_ERR_DUPLICATE_ADDR	No controller status change

The MMIO STATUS register exposes one generic ERROR bit. The C return code distinguishes no-slave/NACK-like failures from invalid command sequence. A future design could add an MMIO-visible ERROR_CAUSE register if firmware needs that distinction.

12. Design Notes from Review

Observation	Resolution
Device ID writable	Documented as intentional simplification. Future extension can add register permissions.
SEND_ADDR transient	Documented. It is retained to show the natural state-machine step and future timing expansion path.
Address mask in i2c_bus_find	Simplified. Bus find now expects a 7-bit address; START passes the already-decoded address.
No NACK distinction in STATUS	Documented as future improvement. Function return codes already distinguish I2C_ERR_NO_SLAVE.

13. Validation Strategy

The tests are written as firmware-like MMIO sequences. They verify both successful transactions and error paths.

Test	Purpose
Device ID read	Verifies common register-read transaction using repeated START.
Register write/readback	Verifies endpoint register pointer and data update.
Multiple endpoints	Verifies bus routing by address.
Missing slave	Verifies no-slave error path.
READ without active transaction	Verifies invalid sequence handling.
STOP without active transaction	Verifies firmware sequencing bug detection.
Disabled controller command	Verifies CONTROL.ENABLE enforcement.
Duplicate address rejected	Verifies bus integrity.
STATUS write-one-to-clear	Verifies sticky status clearing semantics.

14. Build and Run

```
make test
make demo
make clean
```

15. Assumptions and Limitations

- Single I2C master only.
- Multiple slave endpoints are supported.
- 7-bit addressing only.
- Transaction-level modelling only; no SDA/SCL waveform simulation.
- No clock stretching, arbitration, timing model, or multi-master behavior.
- Repeated START is supported.
- Endpoint register pointer convention is used for simple register reads/writes.
- Device ID register is writable as an intentional simplification.
- Generic ERROR status bit is used; detailed cause is available through C return codes.

16. Future Improvements

- Add interrupt enable/status registers and an IRQ output line.
- Add FIFO support for TX and RX.
- Add per-register access permissions for endpoint registers.
- Add an MMIO-visible error-cause register for NACK/invalid-sequence distinction.
- Add command latency and event scheduling for timing-aware simulation.
- Add trace logging hooks for debug